

Assignment 11.3 Ai Assisted Coding

Ht.no: 2303A51988

Batch: 06

Task 1:

Smart Contact Manager (Arrays & Linked Lists)

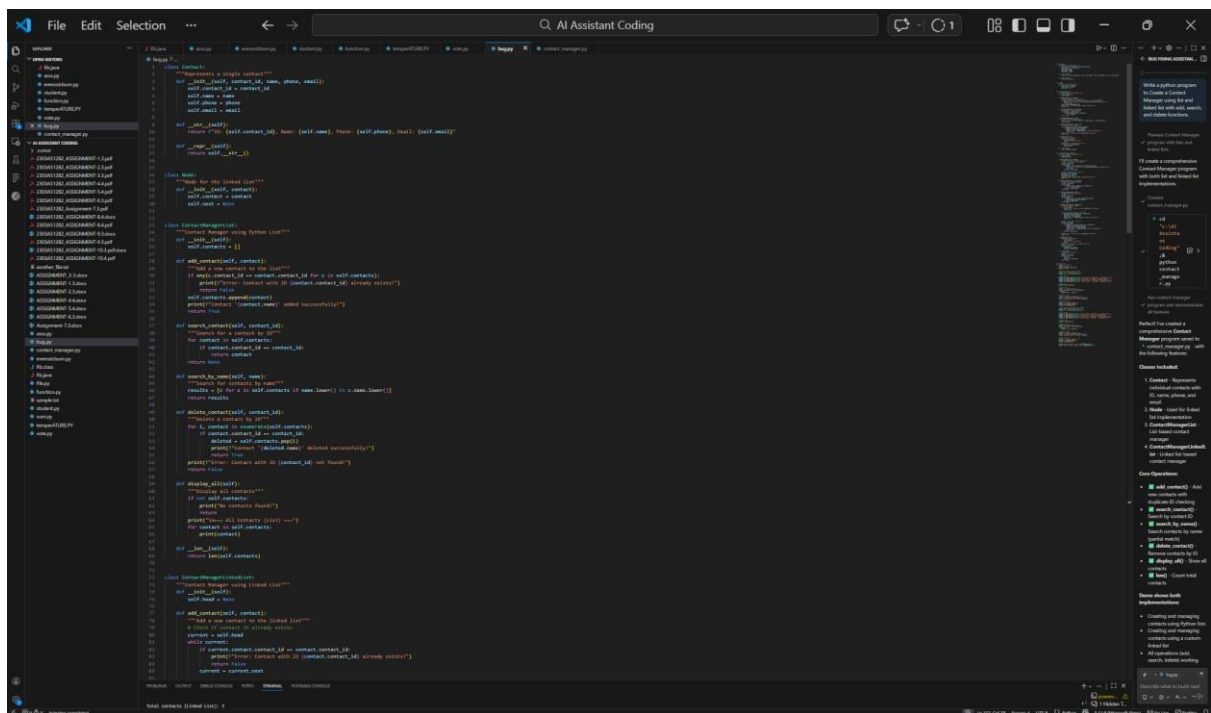
Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Prompt:

Write a python program to Create a Contact Manager using list and linked list with add, search, and delete functions.

Code:



```
class Contact:
    """Represents a single contact"""
    def __init__(self, contact_id, name, phone, email):
        self.contact_id = contact_id
        self.name = name
        self.phone = phone
        self.email = email

class ContactManager:
    """Contact Manager using linked list"""
    def __init__(self):
        self._head = None

    def add_contact(self, contact):
        """Add a new contact to the linked list"""
        if not self._head:
            self._head = contact
            return
        current = self._head
        while current.next:
            current = current.next
        current.next = contact

    def search_contact(self, contact_id):
        """Search for a contact by ID"""
        current = self._head
        while current:
            if current.contact_id == contact_id:
                return current
            current = current.next
        return None

    def search_by_name(self, name):
        """Search for a contact by name"""
        current = self._head
        while current:
            if current.name == name:
                return current
            current = current.next
        return None

    def delete_contact(self, contact_id):
        """Delete a contact by ID"""
        if not self._head:
            return
        if self._head.contact_id == contact_id:
            self._head = self._head.next
            return
        current = self._head
        while current.next:
            if current.next.contact_id == contact_id:
                current.next = current.next.next
                return
            current = current.next.next
        return

    def display_all(self):
        """Display all contacts"""
        if not self._head:
            print("No contacts found")
            return
        current = self._head
        while current:
            print(f"Contact ID: {current.contact_id}, Name: {current.name}, Phone: {current.phone}, Email: {current.email}")
            current = current.next

    def _insert(self, contact):
        """Insert a new contact at the beginning of the linked list"""
        new_contact = Contact(contact_id, name, phone, email)
        new_contact.next = self._head
        self._head = new_contact

    def _delete(self, contact_id):
        """Delete a contact from the linked list"""
        if not self._head:
            return
        if self._head.contact_id == contact_id:
            self._head = self._head.next
            return
        current = self._head
        while current.next:
            if current.next.contact_id == contact_id:
                current.next = current.next.next
                return
            current = current.next.next
        return

    def _search(self, contact_id):
        """Search for a contact by ID"""
        current = self._head
        while current:
            if current.contact_id == contact_id:
                return current
            current = current.next
        return None

    def _search_by_name(self, name):
        """Search for a contact by name"""
        current = self._head
        while current:
            if current.name == name:
                return current
            current = current.next
        return None

    def _delete_by_id(self, contact_id):
        """Delete a contact by ID"""
        if not self._head:
            return
        if self._head.contact_id == contact_id:
            self._head = self._head.next
            return
        current = self._head
        while current.next:
            if current.next.contact_id == contact_id:
                current.next = current.next.next
                return
            current = current.next.next
        return

    def _display_all(self):
        """Display all contacts"""
        if not self._head:
            print("No contacts found")
            return
        current = self._head
        while current:
            print(f"Contact ID: {current.contact_id}, Name: {current.name}, Phone: {current.phone}, Email: {current.email}")
            current = current.next
```

```
File Edit Selection *** Q AI Assistant Coding
# Manager.py
class ContactManager:
    def __init__(self):
        self.contacts = {}

    def add_contact(self, name, phone, email):
        """Add a new contact to the list"""
        if name in self.contacts:
            print(f"Contact with name '{name}' already exists.")
            return False
        self.contacts[name] = {"phone": phone, "email": email}
        print(f"Contact '{name}' added successfully.")
        return True

    def search_contact(self, name):
        """Search for a contact by name"""
        if name in self.contacts:
            return self.contacts[name]
        else:
            print(f"Contact with name '{name}' not found.")
            return None

    def search_by_name(self, name):
        """Search for contacts by name"""
        results = []
        for name, contact in self.contacts.items():
            if name.startswith(name.lower()):
                results.append(contact)
        return results

    def delete_contact(self, name):
        """Delete a contact by name"""
        if name in self.contacts:
            del self.contacts[name]
            print(f"Contact '{name}' deleted successfully.")
            return True
        else:
            print(f"Contact with name '{name}' not found.")
            return False

    def update_contact(self, name, phone, email):
        """Update a contact's phone and email"""
        if name in self.contacts:
            self.contacts[name]["phone"] = phone
            self.contacts[name]["email"] = email
            print(f"Contact '{name}' updated successfully.")
            return True
        else:
            print(f"Contact with name '{name}' not found.")
            return False

    def display_all(self):
        """Display all contacts"""
        if not self.contacts:
            print("No contacts found.")
            return
        for name, contact in self.contacts.items():
            print(f"Name: {name}, Phone: {contact['phone']}, Email: {contact['email']}")

    def __str__(self):
        return str(self.contacts)

# Main Program
if __name__ == "__main__":
    manager = ContactManager()

    # Add contacts
    manager.add_contact("John Doe", "123-456-7890", "john.doe@example.com")
    manager.add_contact("Jane Smith", "987-654-3210", "jane.smith@example.com")
    manager.add_contact("Bob Johnson", "555-111-2222", "bob.johnson@example.com")

    # Search for a contact
    name = input("Enter name to search: ")
    contact = manager.search_contact(name)
    if contact:
        print(f"Found contact: {contact}")
    else:
        print("Contact not found.")

    # Search by name
    name = input("Enter name to search by: ")
    results = manager.search_by_name(name)
    if results:
        for contact in results:
            print(f"Found contact: {contact}")
    else:
        print("No contacts found.")

    # Display all contacts
    manager.display_all()

    # Update a contact
    name = input("Enter name to update: ")
    phone = input("Enter new phone: ")
    email = input("Enter new email: ")
    manager.update_contact(name, phone, email)

    # Delete a contact
    name = input("Enter name to delete: ")
    manager.delete_contact(name)

    # Display all contacts again
    manager.display_all()

    # Exit
    print("Exiting program...")
```

```
File Edit Selection *** Q AI Assistant Coding
# Manager.py
class ContactManager:
    def __init__(self):
        self.contacts = {}

    def add_contact(self, name, phone, email):
        """Add a new contact to the list"""
        if name in self.contacts:
            print(f"Contact with name '{name}' already exists.")
            return False
        self.contacts[name] = {"phone": phone, "email": email}
        print(f"Contact '{name}' added successfully.")
        return True

    def search_contact(self, name):
        """Search for a contact by name"""
        if name in self.contacts:
            return self.contacts[name]
        else:
            print(f"Contact with name '{name}' not found.")
            return None

    def search_by_name(self, name):
        """Search for contacts by name"""
        results = []
        for name, contact in self.contacts.items():
            if name.startswith(name.lower()):
                results.append(contact)
        return results

    def delete_contact(self, name):
        """Delete a contact by name"""
        if name in self.contacts:
            del self.contacts[name]
            print(f"Contact '{name}' deleted successfully.")
            return True
        else:
            print(f"Contact with name '{name}' not found.")
            return False

    def update_contact(self, name, phone, email):
        """Update a contact's phone and email"""
        if name in self.contacts:
            self.contacts[name]["phone"] = phone
            self.contacts[name]["email"] = email
            print(f"Contact '{name}' updated successfully.")
            return True
        else:
            print(f"Contact with name '{name}' not found.")
            return False

    def display_all(self):
        """Display all contacts"""
        if not self.contacts:
            print("No contacts found.")
            return
        for name, contact in self.contacts.items():
            print(f"Name: {name}, Phone: {contact['phone']}, Email: {contact['email']}")

    def __str__(self):
        return str(self.contacts)

# Main Program
if __name__ == "__main__":
    manager = ContactManager()

    # Add contacts
    manager.add_contact("John Doe", "123-456-7890", "john.doe@example.com")
    manager.add_contact("Jane Smith", "987-654-3210", "jane.smith@example.com")
    manager.add_contact("Bob Johnson", "555-111-2222", "bob.johnson@example.com")

    # Search for a contact
    name = input("Enter name to search: ")
    contact = manager.search_contact(name)
    if contact:
        print(f"Found contact: {contact}")
    else:
        print("Contact not found.")

    # Search by name
    name = input("Enter name to search by: ")
    results = manager.search_by_name(name)
    if results:
        for contact in results:
            print(f"Found contact: {contact}")
    else:
        print("No contacts found.")

    # Display all contacts
    manager.display_all()

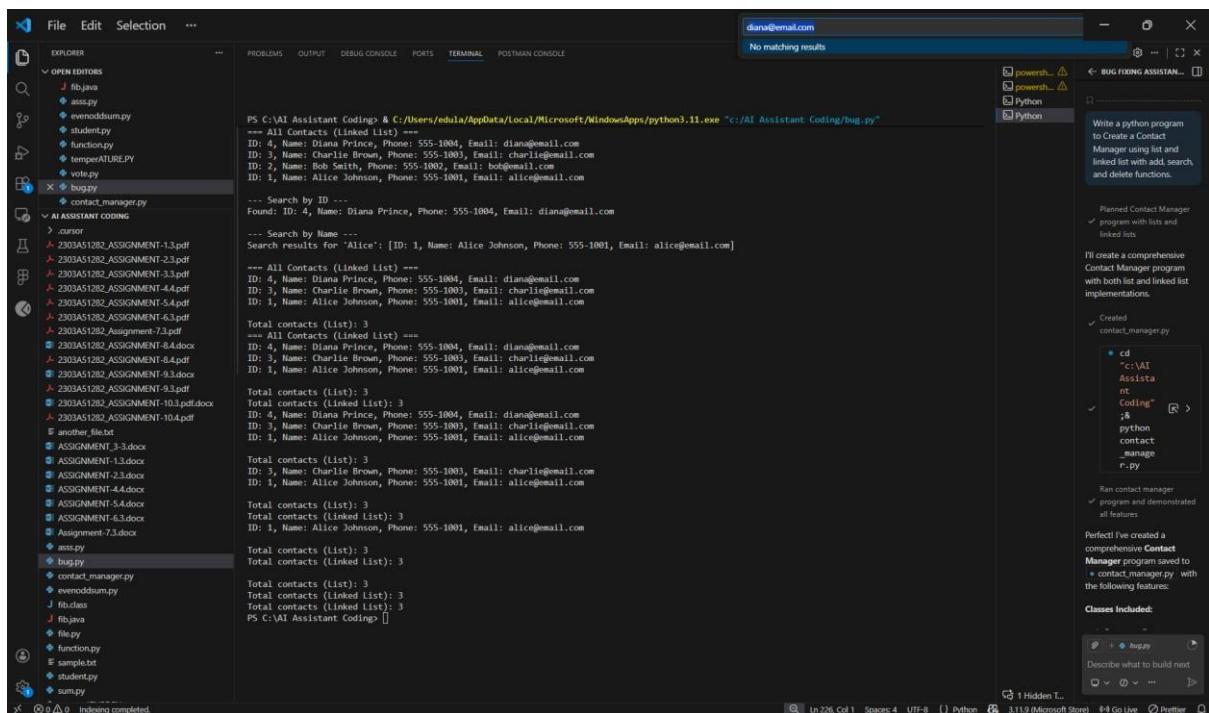
    # Update a contact
    name = input("Enter name to update: ")
    phone = input("Enter new phone: ")
    email = input("Enter new email: ")
    manager.update_contact(name, phone, email)

    # Delete a contact
    name = input("Enter name to delete: ")
    manager.delete_contact(name)

    # Display all contacts again
    manager.display_all()

    # Exit
    print("Exiting program...")
```

Output:



Explanation:

- In an array, adding at the end is fast, but inserting in the middle is slow because elements must shift.
- In a linked list, insertion is fast because no shifting is needed.
- Searching takes the same time in both (you must check each element).
- Deleting in an array is slower due to shifting elements.
- Linked list is better for frequent insertions and deletions.

Task 2:

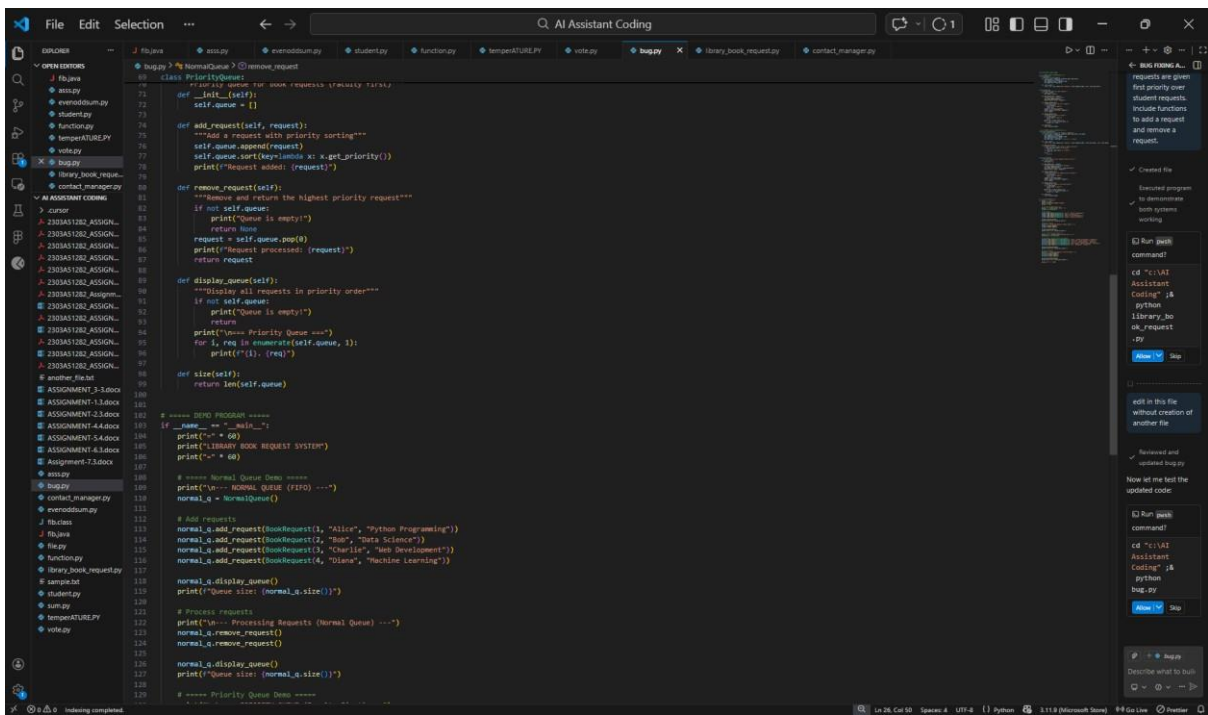
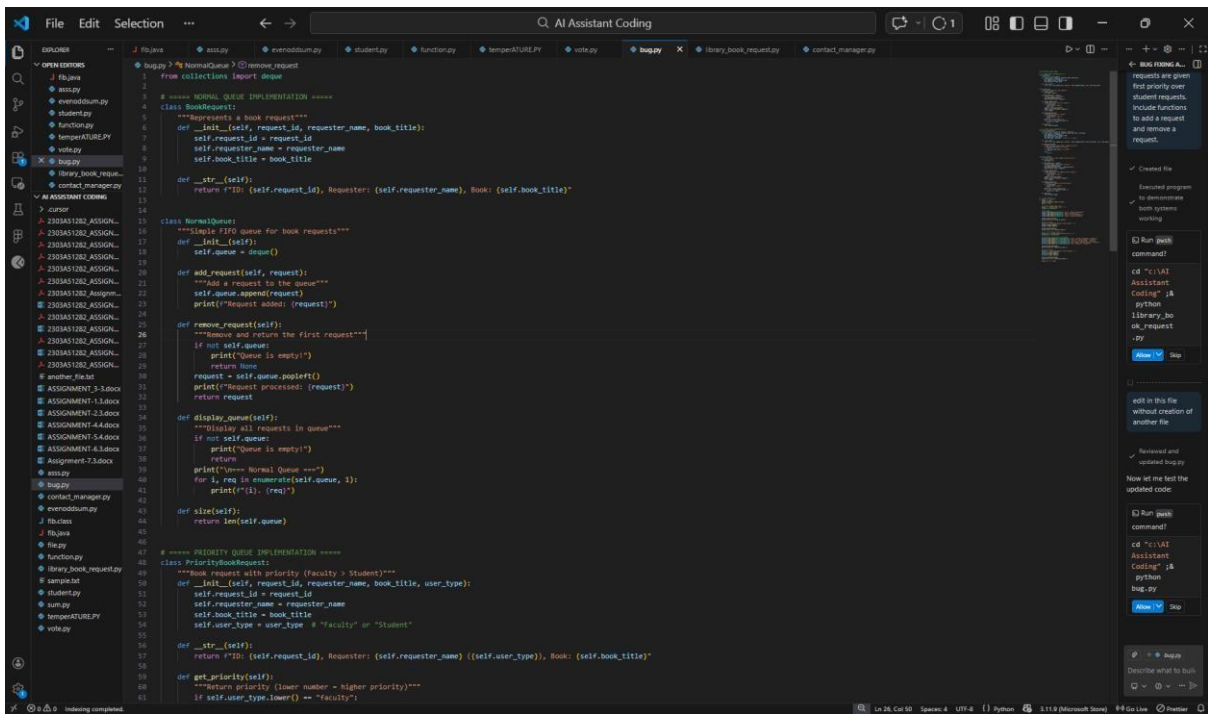
Library Book Search System (Queues & Priority Queues) Scenario

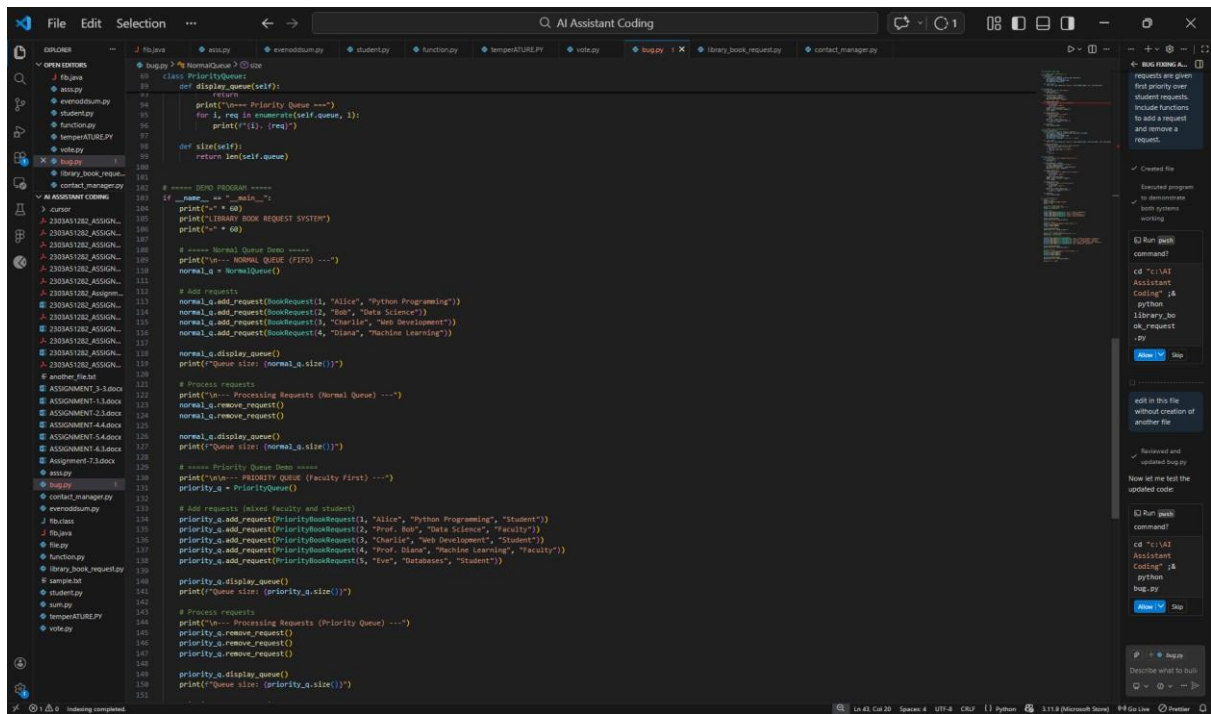
The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Prompt:

Write a Python program for a library book request system. First, make a normal queue where requests are handled in the order they come. Then, make another version where faculty requests are given first priority over student requests. Include functions to add a request and remove a request.

Code:





Output:

```

PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"

=== Priority Queue ===
1. ID: 3, Requester: Charlie (Student), Book: Web Development
2. ID: 5, Requester: Eve (Student), Book: Databases
Queue size: 2

=====
PS C:\AI Assistant Coding>

```

Explanation:

- Queue (FIFO) → First request comes, first served.(If a student requests first, they get the book first.)
- Priority Queue → Faculty requests are served before students, even if they come later.
- enqueue() → Adds a request to the system.
- dequeue() → Removes and processes the next request.

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

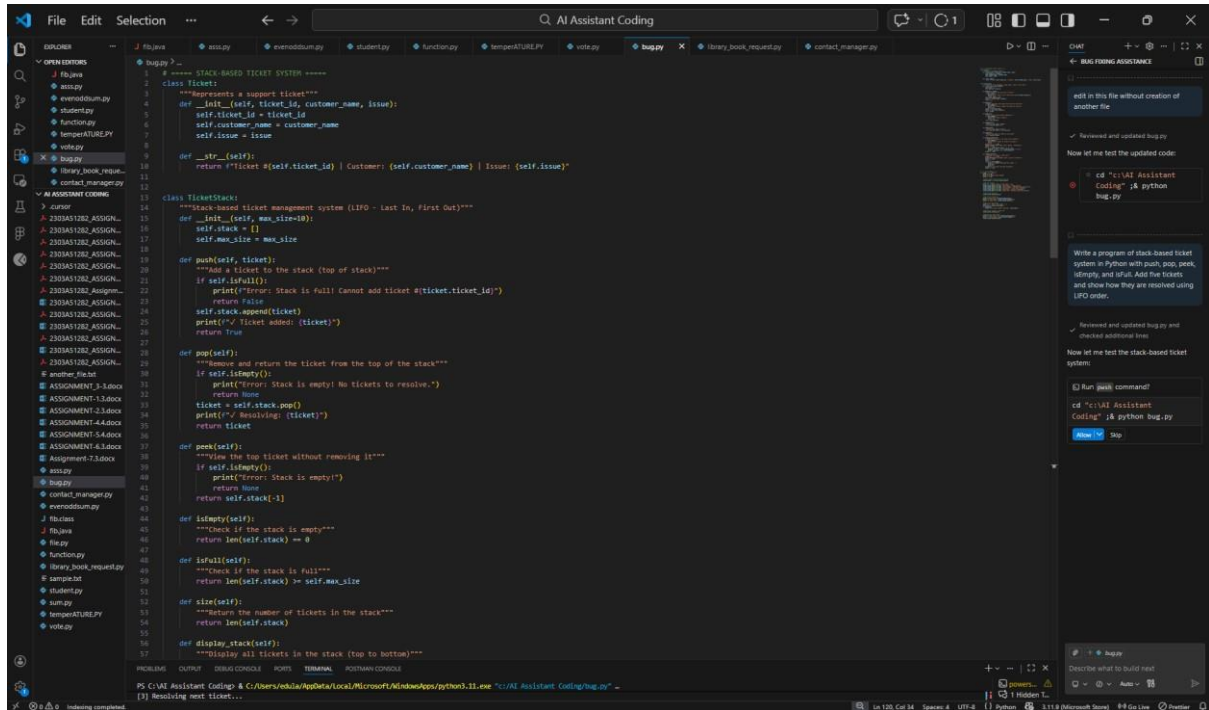
SR University's IT Help Desk receives technical support tickets from students and staff.

While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

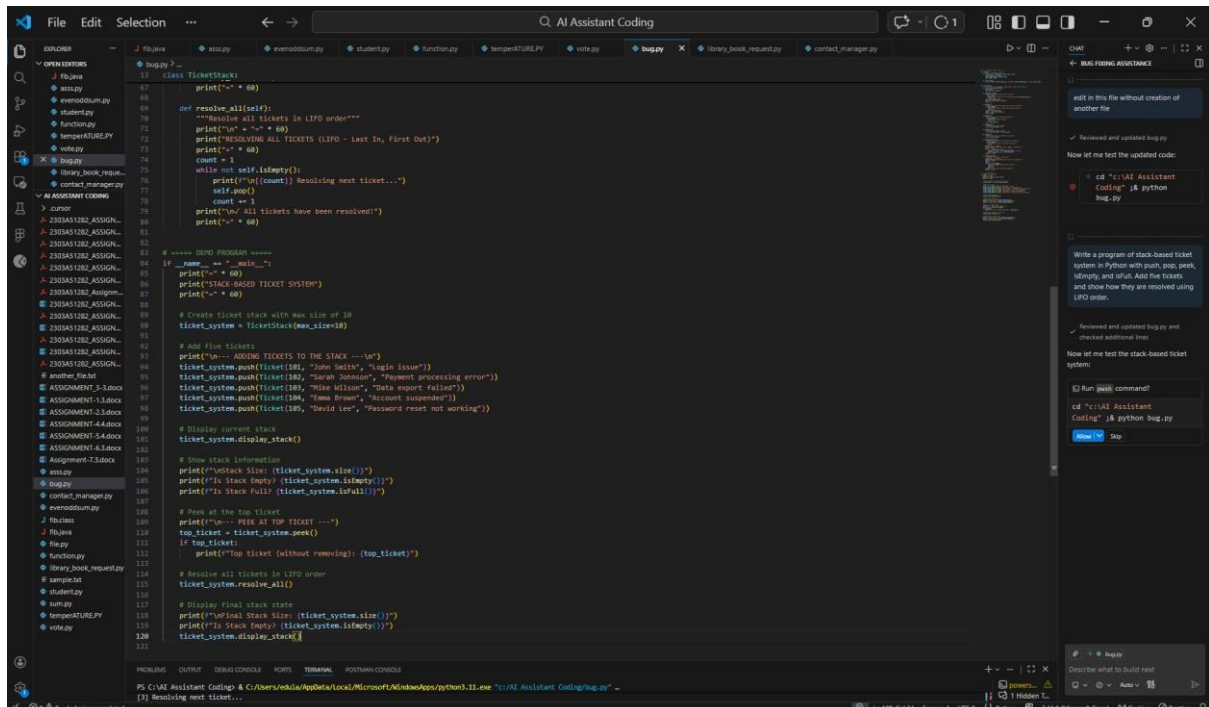
Prompt:

Write a program of stack-based ticket system in Python with push, pop, peek, isEmpty, and isFull. Add five tickets and show how they are resolved using LIFO order.

Code:

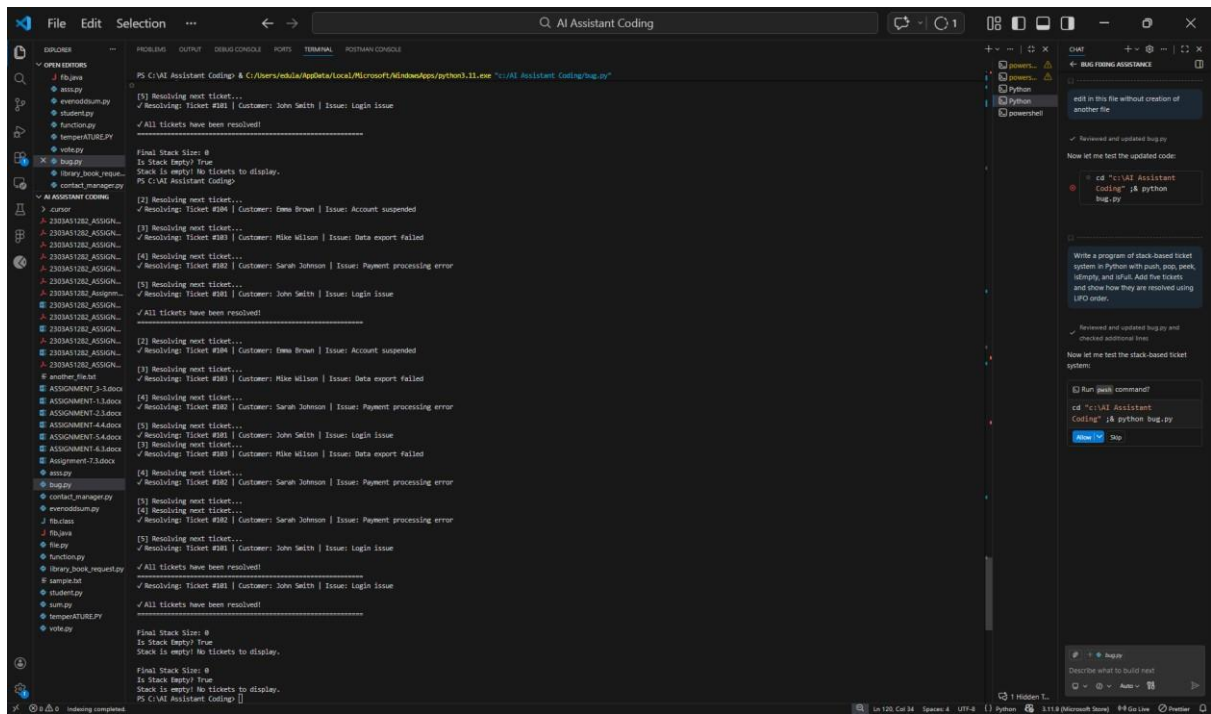


```
1 # ===== STACK-BASED TICKET SYSTEM =====
2 class Ticket:
3     """Represents a support ticket"""
4     def __init__(self, ticket_id, customer_name, issue):
5         self.ticket_id = ticket_id
6         self.customer_name = customer_name
7         self.issue = issue
8
9     def __str__(self):
10         return f"Ticket #{self.ticket_id} | Customer: {self.customer_name} | Issue: {self.issue}"
11
12 class TicketStack:
13     """Stack-based ticket management system (LIFO - Last In, First Out)"""
14     def __init__(self, max_size=10):
15         self.stack = []
16         self.max_size = max_size
17
18     def push(self, ticket):
19         """Add a ticket to the stack (top of stack)"""
20         if self.isFull():
21             print(f"Error: Stack is full! Cannot add ticket #{ticket.ticket_id}")
22             return False
23         self.stack.append(ticket)
24         print(f"/> Ticket added: {ticket}")
25         return True
26
27     def pop(self):
28         """Remove and return the ticket from the top of the stack"""
29         if self.isEmpty():
30             print(f"Error: Stack is empty! No tickets to resolve.")
31             return None
32         ticket = self.stack.pop()
33         print(f"/> Resolving: {ticket}")
34         return ticket
35
36     def peek(self):
37         """View the top ticket without removing it"""
38         if self.isEmpty():
39             print(f"Error: Stack is empty!")
40             return None
41         return self.stack[-1]
42
43     def isEmpty(self):
44         """Check if the stack is empty"""
45         return len(self.stack) == 0
46
47     def isFull(self):
48         """Check if the stack is full"""
49         return len(self.stack) >= self.max_size
50
51     def size(self):
52         """Return the number of tickets in the stack"""
53         return len(self.stack)
54
55     def display_stack(self):
56         """Display all tickets in the stack (top to bottom)"""
57         for ticket in reversed(self.stack):
58             print(f"Ticket: {ticket}")
```



```
1 # ===== STACK-BASED TICKET SYSTEM =====
2 class Ticket:
3     """Represents a support ticket"""
4     def __init__(self, ticket_id, customer_name, issue):
5         self.ticket_id = ticket_id
6         self.customer_name = customer_name
7         self.issue = issue
8
9     def __str__(self):
10         return f"Ticket #{self.ticket_id} | Customer: {self.customer_name} | Issue: {self.issue}"
11
12 class TicketStack:
13     """Stack-based ticket management system (LIFO - Last In, First Out)"""
14     def __init__(self, max_size=10):
15         self.stack = []
16         self.max_size = max_size
17
18     def push(self, ticket):
19         """Add a ticket to the stack (top of stack)"""
20         if self.isFull():
21             print(f"Error: Stack is full! Cannot add ticket #{ticket.ticket_id}")
22             return False
23         self.stack.append(ticket)
24         print(f"/> Ticket added: {ticket}")
25         return True
26
27     def pop(self):
28         """Remove and return the ticket from the top of the stack"""
29         if self.isEmpty():
30             print(f"Error: Stack is empty! No tickets to resolve.")
31             return None
32         ticket = self.stack.pop()
33         print(f"/> Resolving: {ticket}")
34         return ticket
35
36     def peek(self):
37         """View the top ticket without removing it"""
38         if self.isEmpty():
39             print(f"Error: Stack is empty!")
40             return None
41         return self.stack[-1]
42
43     def isEmpty(self):
44         """Check if the stack is empty"""
45         return len(self.stack) == 0
46
47     def isFull(self):
48         """Check if the stack is full"""
49         return len(self.stack) >= self.max_size
50
51     def size(self):
52         """Return the number of tickets in the stack"""
53         return len(self.stack)
54
55     def display_stack(self):
56         """Display all tickets in the stack (top to bottom)"""
57         for ticket in reversed(self.stack):
58             print(f"Ticket: {ticket}")
59
60 # ===== MAIN PROGRAM =====
61 if __name__ == "__main__":
62     print("===== STACK-BASED TICKET SYSTEM =====")
63
64     # Create ticket stack with max size of 10
65     ticket_system = TicketStack(max_size=10)
66
67     # Add five tickets
68     print("> Adding tickets to the stack --->")
69     ticket_system.push(Ticket(101, "John Smith", "Login issue"))
70     ticket_system.push(Ticket(102, "Sarah Johnson", "Payment processing error"))
71     ticket_system.push(Ticket(103, "Mike Wilson", "Data report failure"))
72     ticket_system.push(Ticket(104, "Emma Brown", "Account suspended"))
73     ticket_system.push(Ticket(105, "David Lee", "Password reset not working"))
74
75     # Display current stack
76     ticket_system.display_stack()
77
78     # Show stack information
79     print(f"> Stack Size: {ticket_system.size()}")
80     print(f"> Is Stack Empty? {ticket_system.isEmpty()}")
81     print(f"> Is Stack Full? {ticket_system.isFull()}")
82
83     # Peek at the top ticket
84     print(f"> Peek at top ticket --->")
85     top_ticket = ticket_system.peek()
86     if top_ticket:
87         print(f"> Top ticket (without removing): {top_ticket}")
88
89     # Resolve all tickets in LIFO order
90     ticket_system.resolve_all()
91
92     # Display final stack state
93     print(f"> Final Stack Size: {ticket_system.size()}")
94     print(f"> Is Stack Empty? {ticket_system.isEmpty()}")
95     ticket_system.display_stack()
```

Output:



Explanation:

The program uses a stack to manage help desk tickets.

A stack works in last in, first solved order.

When a new ticket is raised, it is added to the top.

When solving a ticket, the most recent one is handled first.

The program can also check if there are no tickets left or if the stack is full.

Task 4:

Hash Table

Objective

To implement a Hash Table and understand collision handling.

Prompt:

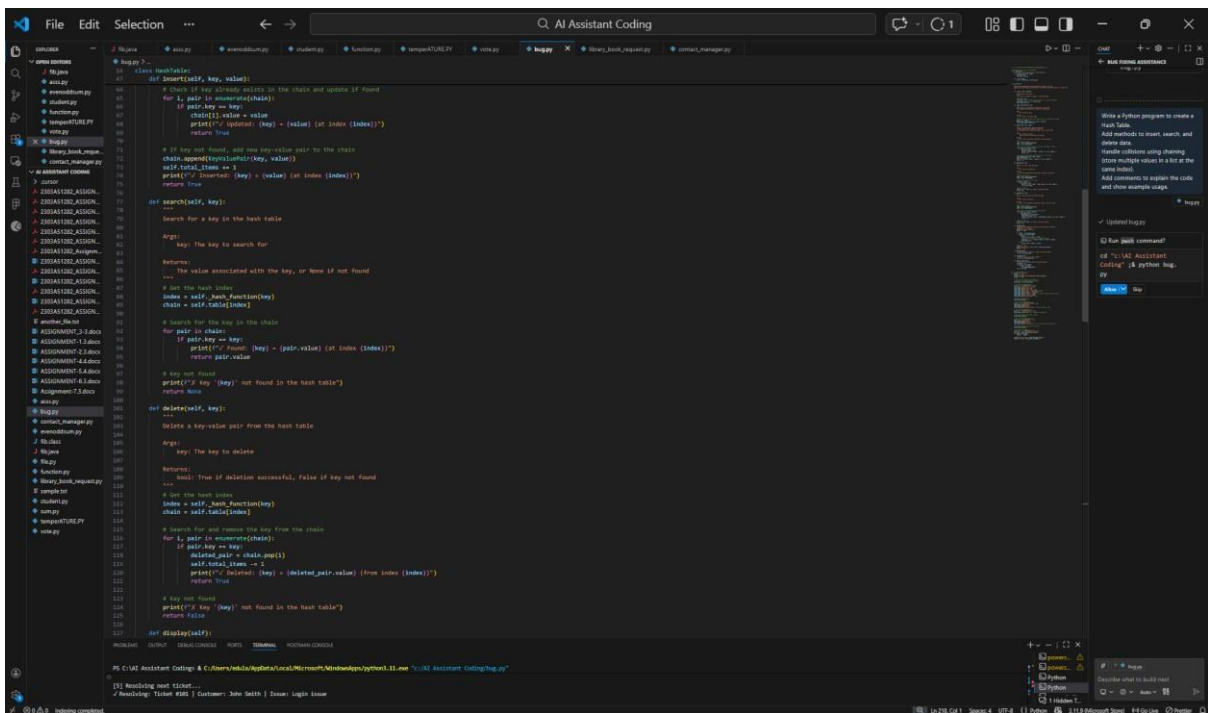
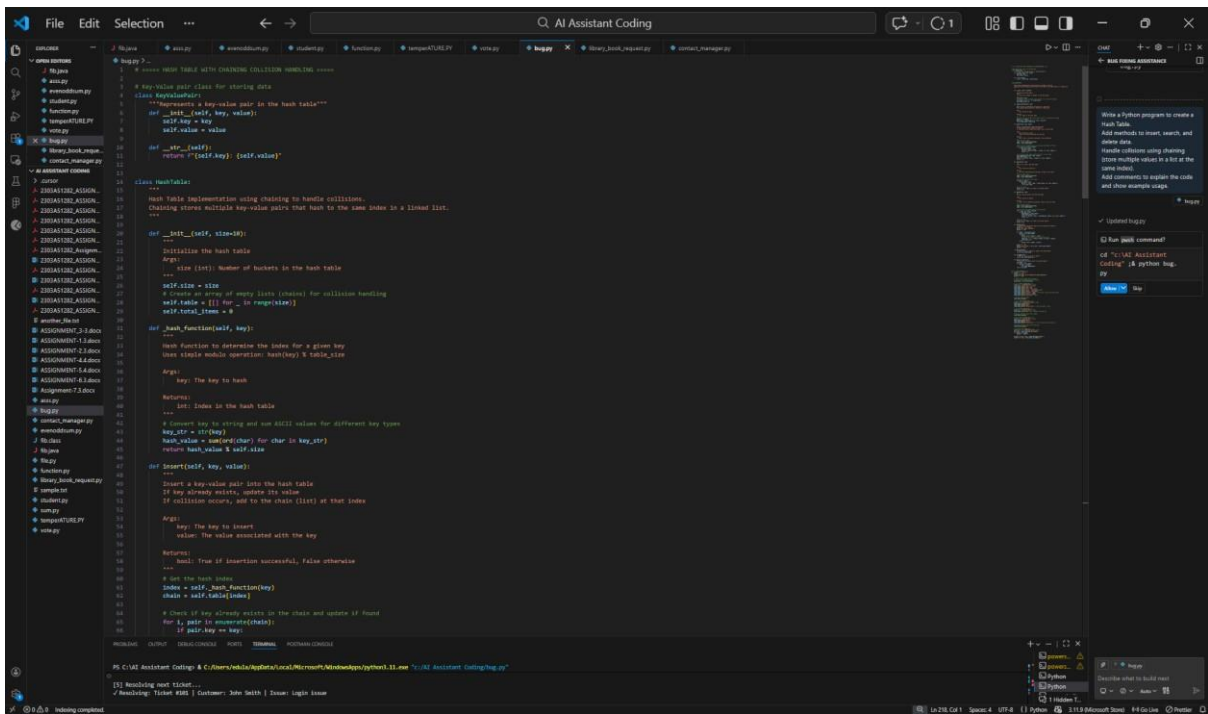
Write a Python program to create a Hash Table.

Add methods to insert, search, and delete data.

Handle collisions using chaining (store multiple values in a list at the same index).

Add comments to explain the code and show example usage.

Code:



```
def get_all_items(self):
    """Returns all key-value pairs in the hash table"""
    all_items = []
    for chain in self.table:
        for pair in chain:
            all_items.append(pair)
    return all_items

# ===== RUNNING PROGRAM =====
if __name__ == '__main__':
    print("\n ~~~~~")
    print("HASH TABLE WITH CHAINING COLLISION HANDLING")
    print("\n ~~~~~")

    # Create a hash table with 5 buckets
    # (will also be demonstrated collisions)
    hash_table = HashTable(5)

    # ===== INSERT OPERATIONS =====
    print("\n--- INSERTING DATA ---")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")
    hash_table.insert("email", "alice@gmail.com")
    hash_table.insert("name", "Alice")
    hash_table.insert("phone", 555-1234)
    hash_table.insert("country", "USA") # Note key collides with other keys

    # Display the hash table
    hash_table.display()

    # ===== SEARCH OPERATIONS =====
    print("\n--- SEARCHING FOR DATA ---")
    hash_table.search("name")
    hash_table.search("city")
    hash_table.search("unknown_key") # Key that doesn't exist

    # ===== UPDATE OPERATIONS =====
    print("\n--- UPDATING DATA ---")
    hash_table.update("age", 31) # Update existing key

    # Display the hash table after update
    hash_table.display()

    # ===== DELETE OPERATIONS =====
    print("\n--- DELETING DATA ---")
    hash_table.delete("email")
    hash_table.delete("city")
    hash_table.delete("nonexistent") # Try to delete non-existent key

    # Display the hash table after deletions
    hash_table.display()

    # ===== GET ALL ITEMS =====
    print("\n--- ALL REMAINING ITEMS ---")
    all_items = hash_table.get_all_items()
    for item in all_items:
        print(f"Item: {item}")

    print("\nFinal items: (hash_table.get_all_items())")
    print(f"Is empty: {hash_table.is_empty()}")
```

Output:

```
PS C:\AI Assistant Codings & C:\Users\eduka\AppData\Local\Microsoft\WindowsApps\python11.exe "C:\AI Assistant Codings\bug.py"
Index 0: age: 30 -> city: New York -> department: IT
Index 1: name: Alice -> salary: 75000
Index 2: phone: 555-1234 -> country: USA
Index 3: [empty]
Total items in hash table: 6

--- SEARCHING FOR DATA ---
Found: name = Alice (at index 1)
Found: age = 30 (at index 0)
Key 'unknown_key' not found in the hash table

--- UPDATING DATA ---
Updated: age = 31 (at index 0)

HASH TABLE CONTENTS
Index 0: email: alice@gmail.com
Index 1: age: 31 -> city: New York -> department: IT
Index 2: name: Alice -> salary: 75000
Index 3: phone: 555-1234 -> country: USA
Index 4: [empty]
Total items in hash table: 6

--- DELETING DATA ---
Deleted: email = alice@gmail.com (from index 0)
Deleted: city = New York (from index 1)
Key 'nonexistent' not found in the hash table

HASH TABLE CONTENTS
Index 0: [empty]
Index 1: age: 31 -> department: IT
Index 2: name: Alice -> salary: 75000
Index 3: phone: 555-1234 -> country: USA
Index 4: [empty]
Total items in hash table: 4

--- ALL REMAINING ITEMS ---
age: 31
department: IT
name: Alice
salary: 75000
phone: 555-1234
country: USA
Is empty: False
PS C:\AI Assistant Codings & C:\Users\eduka\AppData\Local\Microsoft\WindowsApps\python11.exe "C:\AI Assistant Codings\bug.py"
```

Explanation:

- A Hash Table stores data using a key and value.
- A hash function decides where to store the data.
- Sometimes two keys go to the same place. This is called a collision.
- To solve collisions, we use chaining, meaning we store multiple items in a list at the same index.
- The program should allow adding, finding, and removing data correctly.

Task 5:

Real-Time Application Challenge Scenario

Design a Campus Resource Management System with the following features:

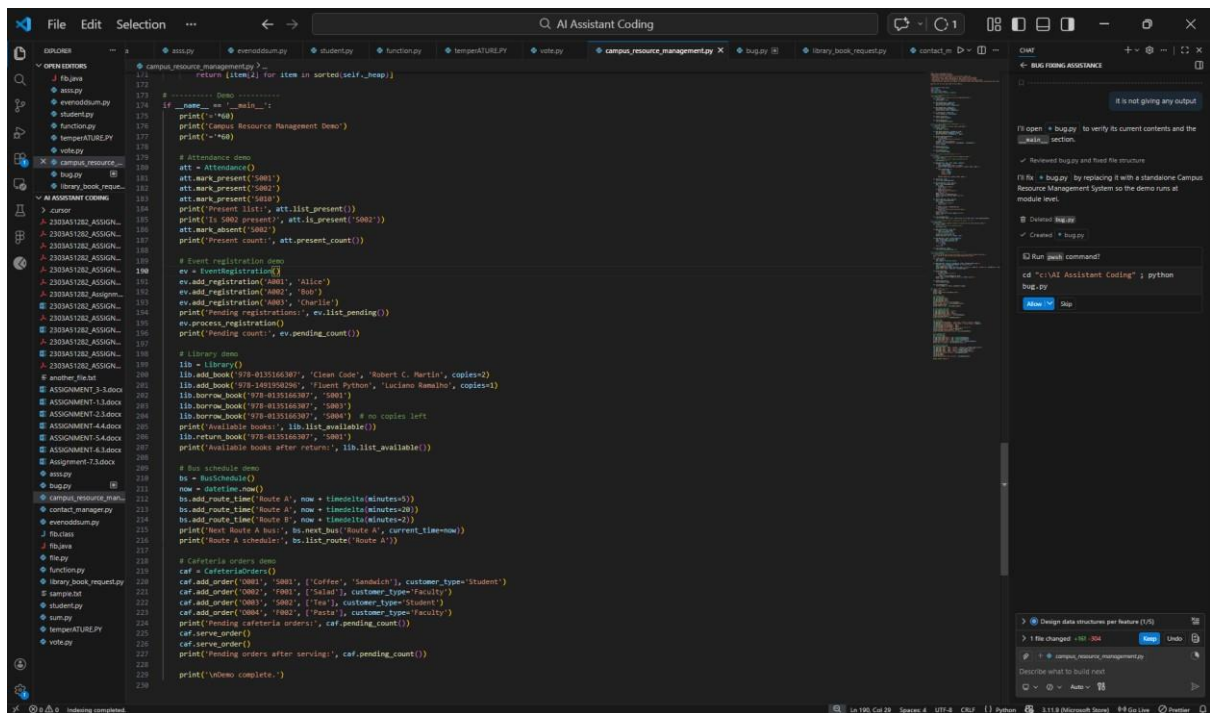
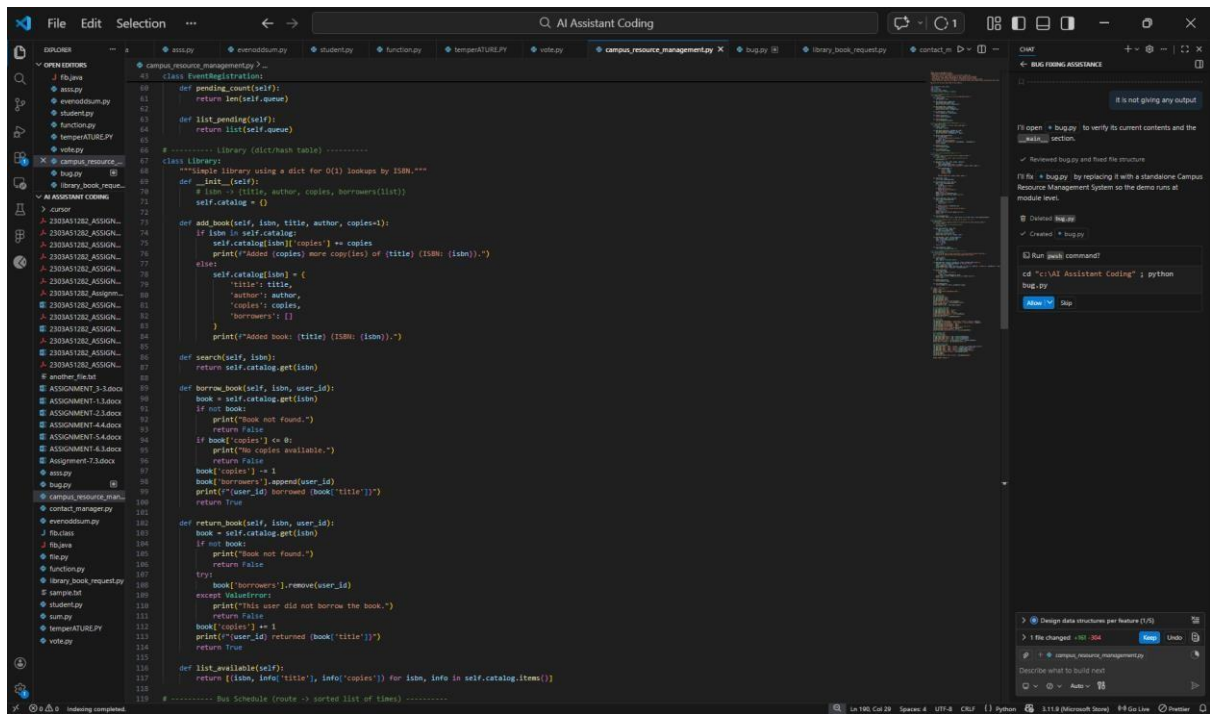
- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Prompt:

Create a Campus Resource Management System in Python. For each feature (Attendance, Event Registration, Library, Bus Schedule, Cafeteria Orders), choose the best data structure

Code:

```
File Edit Selection ... AI Assistant Coding
campus_resource_management.py
1 """
2 Campus Resource Management System
3 Features and chosen data structures:
4 - Attendance: set (O(1) add/check) to track present student IDs
5 - Event Registration: deque (FIFO queue) for fair registration processing
6 - Library: dict (O(1) lookup) mapping book ID to book record for fast lookup
7 - Bus Schedule: dict of route -> sorted list of departure times (list kept sorted)
8 - Cafeteria Orders: heap (priority queue) to prioritize faculty over students while preserving arrival order
9
10 Run this file to see a small demo of each feature.
11 """
12
13 from collections import deque
14 import heapq
15 import sys
16 from bisect import bisect
17 from datetime import datetime, timedelta
18
19 # ===== Attendance (set) =====
20 class Attendance:
21     """Track attendance using a set for O(1) add/remove/check."""
22     def __init__(self):
23         self.present = set()
24
25     def mark_present(self, student_id):
26         self.present.add(student_id)
27         print(f"Marked present: {student_id}")
28
29     def mark_absent(self, student_id):
30         self.present.discard(student_id)
31         print(f"Marked absent: {student_id}")
32
33     def is_present(self, student_id):
34         return student_id in self.present
35
36     def present_count(self):
37         return len(self.present)
38
39     def list_present(self):
40         return sorted(self.present)
41
42 # ===== Event Registration (FIFO queue) =====
43 class EventRegistration:
44     """Register attendees in arrival order using deque."""
45     def __init__(self):
46         self.queue = deque()
47
48     def add_registration(self, attendee_id, name):
49         self.queue.append((attendee_id, name))
50         print(f"Registered: {attendee_id} - {name}")
51
52     def process_registration(self):
53         if not self.queue:
54             print("No registrations to process.")
55             return None
56         attendee = self.queue.popleft()
57         print(f"Processing registration: {attendee[0]} - {attendee[1]}")
58         return attendee
59
60     def pending_count(self):
61         return len(self.queue)
62
63 # ===== Library (dict) =====
64 class Library:
65     """Manage library books using a dict for O(1) lookup."""
66     def __init__(self):
67         self.books = {}
68
69     def add_book(self, book_id, title, author):
70         self.books[book_id] = {"title": title, "author": author}
71
72     def get_book(self, book_id):
73         return self.books.get(book_id)
74
75     def list_books(self):
76         return list(self.books.values())
77
78 # ===== Bus Schedule (dict of sorted lists) =====
79 class BusSchedule:
80     """Manage bus routes using a dict of sorted lists for departure times."""
81     def __init__(self):
82         self.routes = {}
83
84     def add_route(self, route_id, route_name, departure_times):
85         self.routes[route_id] = {"name": route_name, "times": sorted(departure_times)}
86
87     def get_route(self, route_id):
88         return self.routes.get(route_id)
89
90     def list_routes(self):
91         return list(self.routes.values())
92
93 # ===== Cafeteria Orders (heap) =====
94 class CafeteriaOrders:
95     """Manage cafeteria orders using a heap for prioritization (priority queue)."""
96     def __init__(self):
97         self.orders = []
98
99     def add_order(self, order_id, priority, name):
100         # Priority is negative for max-heap behavior
101         heapq.heappush(self.orders, (-priority, order_id, name))
102
103     def get_order(self, order_id):
104         for i, (priority, order_id, name) in enumerate(self.orders):
105             if order_id == order_id:
106                 return (priority, name)
107         return None
108
109     def list_orders(self):
110         return [(priority, name) for priority, order_id, name in self.orders]
111
112 # ===== Main Program =====
113 if __name__ == "__main__":
114     # Create instances of each module
115     attendance = Attendance()
116     event_registration = EventRegistration()
117     library = Library()
118     bus_schedule = BusSchedule()
119     cafeteria_orders = CafeteriaOrders()
120
121     # Demo of each feature
122     print("=== Attendance Demo ===")
123     attendance.mark_present(101)
124     attendance.mark_absent(102)
125     print(f"Present count: {attendance.present_count()}")
126     print(f"List present: {attendance.list_present()}")
127
128     print("=== Event Registration Demo ===")
129     event_registration.add_registration(201, "Alice")
130     event_registration.add_registration(202, "Bob")
131     attendee = event_registration.process_registration()
132     print(f"Pending count: {event_registration.pending_count()}")
133
134     print("=== Library Demo ===")
135     library.add_book(301, "Python 101", "John Doe")
136     book = library.get_book(301)
137     print(f"Book: {book}")
138     print(f"List books: {library.list_books()}")
139
140     print("=== Bus Schedule Demo ===")
141     bus_schedule.add_route(401, "Route A", [8:00, 9:00, 10:00])
142     route = bus_schedule.get_route(401)
143     print(f"Route: {route}")
144     print(f"List routes: {bus_schedule.list_routes()}")
145
146     print("=== Cafeteria Orders Demo ===")
147     cafeteria_orders.add_order(501, 1, "Faculty Order")
148     cafeteria_orders.add_order(502, 2, "Student Order")
149     (priority, name) = cafeteria_orders.get_order(501)
150     print(f"Order: {priority} {name}")
151     print(f"List orders: {cafeteria_orders.list_orders()}")
152
153     # Design data structures per feature (1/5)
154     # 1 file changed - 161 lines
155     # campus_resource_management.py
156     # Describe what to build next
```



Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL POSTMAN CONSOLE

PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"
Is empty: False
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/campus_resource_management.py"

=====
Campus Resource Management Demo
=====

Marked present: 5001
Marked present: 5002
Marked present: 5010
Present list: ['5001', '5002', '5010']
Is 5002 present? True
Marked absent: 5002
Present count: 2
Registered: A001 - Alice
Registered: A002 - Bob
Registered: A003 - Charlie
Pending registrations: [('A001', 'Alice'), ('A002', 'Bob'), ('A003', 'Charlie')]
Processed registration: A001 - Alice
Pending count: 2
Added book: Clean Code (ISBN: 978-0135166307).
Added book: Fluent Python (ISBN: 978-1491958296).
5001 borrowed Clean Code
5003 borrowed Clean Code
No copies available.
Available books: [('978-0135166307', 'Clean Code', 0), ('978-1491958296', 'Fluent Python', 1)]
5001 returned Clean Code
Available books after return: [('978-0135166307', 'Clean Code', 1), ('978-1491958296', 'Fluent Python', 1)]
Added bus time for Route A: 2026-02-18 10:42:24.367227
Added bus time for Route A: 2026-02-18 10:57:24.367227
Added bus time for Route B: 2026-02-18 10:39:24.367227
Next Route A bus: 2026-02-18 10:42:24.367227
Route A schedule: [datetime.datetime(2026, 2, 18, 10, 42, 24, 367227), datetime.datetime(2026, 2, 18, 10, 57, 24, 367227)]
Order added: 0001 (Student)
Order added: 0002 (Faculty)
Order added: 0003 (Student)
Order added: 0004 (Faculty)
Pending cafeteria orders: 4
Serving order: 0002 (Faculty)
Serving order: 0004 (Faculty)
Pending orders after serving: 2

Demo complete.
PS C:\AI Assistant Coding>
```

Explanation:

Library Book Borrowing using a queue:

- The queue stores student names who request a book.
- When a student requests a book, we use `enqueue()` to add them to the queue.
- When a book becomes available, we use `dequeue()` to give it to the first student in line.
- This ensures fairness because the first requester gets the book first.